CS 300 Portfolio: ABCU Course Advising Program

This repository holds two artifacts from my data structures and algorithms coursework:

- `Project_One_Runtime_and_Memory_Analysis.md` : my runtime and memory analysis of a vector, a hash table, and a binary search tree for storing Computer Science course data, along with my recommendation.
- `Project_Two_Course_List_Code.cpp` : the working code that stores the course list in a binary search tree and prints it in alphanumeric order with an in-order traversal.

Reflection

What was the problem you were solving in the projects for this course? ABC University's advisors needed a program that could load the Computer Science course catalog from a file, print the whole list in order, and look up a single course along with its prerequisites. Project One asked me to design and compare three data structures for that job (a vector, a hash table, and a binary search tree) and recommend one. Project Two asked me to build the recommended structure into a working C++ program.

How did you approach the problem? Consider why data structures are important to understand. I started by writing pseudocode for each structure's load, print, and search operations, then worked out the Big O runtime and memory cost of each one by hand. That comparison is what let me make an informed recommendation instead of a guess: the binary search tree won because it keeps courses in sorted order as they're inserted, so printing the list costs O(n) with no sort step, while the vector and hash table both need an O(n log n) sort first. Understanding data structures matters because the same problem, storing some records, listing them, and looking one up, can run in O(n), O(n log n), or O(1) depending entirely on how the data is organized, and that difference is invisible until you actually analyze it.

How did you overcome any roadblocks you encountered while going through the activities or project? The trickiest part was validating the input file correctly: making sure every course had at least a number and a name, and that every prerequisite listed on a line was itself a real course somewhere in the file, without leaving the tree half-loaded if an error turned up partway through. I solved it by fully reading and checking the file in two passes before building a single Course object, so the tree is only touched once the whole file is known to be valid. Removing a node with two children in the earlier binary search tree assignment was another sticking point; tracing through it by hand with a small example

helped me see that copying up the in-order successor and then deleting it from its own subtree was the way to keep the tree correctly ordered.

How has your work on this project expanded your approach to designing software and developing programs? I'm now in the habit of comparing data structures on paper before writing any code, instead of defaulting to whatever is most familiar. Working out the runtime of each option up front made it obvious that the "simplest" structure to code, the vector, was not actually the best one for this job.

How has your work on this project evolved the way you write programs that are maintainable, readable, and adaptable? I separated the course data, the tree structure, and the menu logic into their own functions and classes, commented each function with what it does and why, and validated input in one place instead of scattering checks throughout the program. That structure means the course lookup or the file loader could be reused or swapped out, for a different data structure, for example, without having to rewrite the menu or the Course definition around it.